run_proxy_bank.sh — HelixQA proxy bank runner

Revision: 1 **Last modified:** 2026-07-01T09:10:00Z **Authority:** Inherits `constitution/Constitution.md` per §11.4.35. §11.4.18 companion doc for `tools/helixqa/runner/run_proxy_bank.sh`.

Overview

`run_proxy_bank.sh` drives the HelixQA proxy test bank (`tools/helixqa/banks/proxy.yaml` + the per-transport slices under `banks/routes/`) against the **live** proxy data plane and captures real per-route evidence (`result.json` + response codes + sink-side cache log).

It is **anti-bluff by construction** (§11.4 / §11.4.27): the bank is executed by the real `helixqa http` binary through the running proxy, and the cache-HIT assertion is taken from Squid's own `access.log` (`TCP_*HIT`, §11.4.69 sink-side), never from forgeable client headers.

Prerequisites

- The proxy stack is up: HTTP forward on `:53128`, SOCKS5 on `:51080` (`./start`, rootless Podman — §11.4.161).
- To **execute** the bank, the `helixqa` binary must be buildable from `submodules/helix_qa`. That currently requires six own-org sibling modules vendored under `submodules/` (SSH, §11.4.28(C) / §11.4.36): `doc_processor`, `llm_orchestrator`, `llm_provider`, `llms_verifier`, `vision_engine`, `security`. Until they are present the runner takes the honest SKIP path (below) — it never fakes a PASS.
- Optional: a prebuilt binary via `HELIXQA_BIN=/path/to/helixqa` skips the build.

Usage

```
bash tools/helixqa/runner/run_proxy_bank.sh
HELIXQA_BIN=/path/to/helixqa bash tools/helixqa/runner/
        run_proxy_bank.sh
```

Runs under `GOMAXPROCS=2 nice -n 19 ionice -c 3` caps internally (§12.6).

Exit codes

Code	Meaning
0	every route bank PASSed against the live proxy
1	a route FAILed (real product defect)
3	honest §11.4.3 SKIP — the `helixqa` harness could not be built (blocker named + captured)

Edge cases

- **Harness unbuildable** (the current state): the runner writes `qa-results/helixqa/<run-ts>/SKIP.md` + `skip.json` naming the exact missing sibling modules (captured `go build` output — §11.4.6), then exits `3`. This is a SKIP of the *HelixQA build*, not a proxy failure: the proxy data plane is independently proven working (executor-client 204/200/204).
- **No container runtime** to read `proxy-squid:/var/log/squid/access.log`: the cache route records a sink-side-unavailable note rather than asserting a HIT.
- **Proxy down**: routes FAIL (exit 1) — a real defect, not a SKIP.

Internal behaviour

1. Resolve/verify the six sibling modules; if any missing → `emit_skip` → exit 3.
2. Else `go build ./cmd/helixqa`; on build failure → SKIP (build-failed) → exit 3.
3. For each route bank, invoke `helixqa http --bank <b> --base-url <url> --json` with `HTTP_PROXY`/`HTTPS_PROXY` pointed at the proxy, capturing `result.json`.
4. Cache route: read `proxy-squid` access log for `TCP_*HIT` as sink-side proof.
5. Aggregate PASS/FAIL/SKIP; write evidence under `qa-results/helixqa/<run-ts>/`.

Related scripts

- `tools/helixqa/banks/proxy.yaml` — canonical 6-case bank.
- `tools/helixqa/banks/routes/*.yaml` — per-`--base-url` execution slices.

- `challenges/scripts/run_proxy_challenges.sh` — the sibling Challenge bank (independent of the HelixQA harness; runs today with 2 PASS / 1 SKIP).
- `tests/comprehensive-test.sh` — the canonical live-proxy verification suite.

Last verified

2026-07-01 — ran the runner against the live proxy; produced the honest SKIP (exit 3, missing-sibling-modules blocker) at `qa-results/helixqa/20260701T090023Z/`.